

Connectors, Timers and Subprocesses

Mgr. Marián Macik, Principal Software Quality Engineer

April 2026

Topics Covered

- ❑ Connectors
 - ❑ What is a Connector?
 - ❑ Example #1 - Email Connector configuration
 - ❑ Example #2 - REST Connector configuration
- ❑ Timer Nodes
 - ❑ Start, Catch intermediate
 - ❑ Boundary intermediate
 - ❑ Example #3 - Timers
- ❑ Reusable Subprocess/Call Activity
 - ❑ Including passing of data between main process and subprocess
 - ❑ Example #4: Reusable Subprocess/Call Activity
 - ❑ Running independent “subprocess” using signals

Connector

- ❑ Connector is a reusable building block that performs the integration with an external system/service.
- ❑ Connectors are available out-of-the-box (OOTB) and come with Connector Templates which customize how a BPMN element is shown, and how it can be configured by process developers.
- ❑ Custom connector can be created in case a connector for such use case does not exist yet.
- ❑ Plenty of connectors available at [Camunda Marketplace](#).
- ❑ When implementing a process, just create a task and change its type to the desired connector. This will apply the connector template and will show which parameters are needed.

SMTP Servers

- ❑ <https://elasticemail.com/>
 - ❑ Start -> Integrate your app or website -> Connect to SMTP API -> Create SMTP Credentials
 - ❑ Allows sending from verified emails (Settings -> Domains -> Manage Domains -> Email Verification)
 - ❑ Can send only to the same email address used for registration
 - ❑ Server: smtp.elasticemail.com
 - ❑ Port: 2525
 - ❑ Use TLS
- ❑ <https://resend.com>
 - ❑ Generate API KEY
 - ❑ Settings -> SMTP for credentials
 - ❑ From email must have **@resend.dev**
 - ❑ Allows sending email just to the email used for registration
 - ❑ Server: smtp.resend.com
 - ❑ Port: 465
 - ❑ Use SSL
- ❑ <https://seznam.cz> (real email account)
 - ❑ [Configuration](#)
 - ❑ From email must be the email account

Example #1: Email Connector Configuration

- ❑ Create the process definition with email connector
- ❑ Assign a start form to it with fields:
 - ❑ **From** of type **Text field** with key **senderEmail**
 - ❑ **To** of type **Text field** with key **receiverEmail**
 - ❑ **Subject** of type **Text field** with key **subject**
 - ❑ **Body** of type **Text field** with key **body**
- ❑ Configure email connector



From

To

Subject

Body

Authentication

Authentication

Simple

Specify the Email authentication strategy.

Username *fx*

your_email@email.com|

Email password *fx*

your_password

Protocol

Email Protocol

SMTP

SMTP Host *fx*

smtp.elasticemail.com

SMTP Port *fx*

2525

Cryptographic protocol

TLS

* Configuration depends on the chosen SMTP server

Example #1: Email Connector Configuration

- ❑ Configure data mappings for email connector
- ❑ You can optionally set a result variable which will contain additional data after the email is sent

Output mapping ● ▼

Result variable

Name of variable to store the response in

Result expression *fx*

Expression to map the response into process variables

- ❑ Start the process by filling in the start form and verify that you received the email

Send Email ● ▼

From *fx*

To *fx*

Cc *fx*

Bcc *fx*

Headers *fx*

Subject *fx*

ContentType

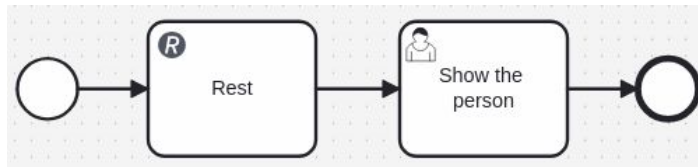
Email Text Content *fx*

Attachment *fx*

Email's attachment. e.g., =[document1, document2]

Example #2: REST Connector Configuration

- ❑ Create the following process definition



- ❑ Assign a start form to it with one field with key **url** of type **Text field**

URL to make request to

- ❑ Assign a form to the Show the person user task, it will have fields:

- ❑ **Name** of type **Text field** with key **person.name**
- ❑ **Age** of type **Number** with key **person.age**

Person

Name

Age

- ❑ Configure Rest connector

HTTP endpoint

Method

GET

URL *fx*

= url

Output mapping

Result variable

Name of variable to store the response in

Result expression *fx*

```
= {person:
  {name: response.body.name,
   age: response.body.age}
}
```

Expression to map the response into process variables

Notice the blue *fx* (click on it to activate), this means a reference to a process variable is used, not the value itself

Example #2: REST Connector Configuration

- ❑ Configure Show the person user task **Inputs**



The screenshot shows the 'Inputs' configuration for a variable named 'person'. It includes a 'Local variable name' field with the value 'person' and a 'Variable assignment value' field with the value '= person'. The 'Variable assignment value' field is preceded by an equals sign and a small 'fx' icon.

- ❑ You can now deploy and run the process, provide a URL in the start form, e.g from [Mocka](#), which will return for example

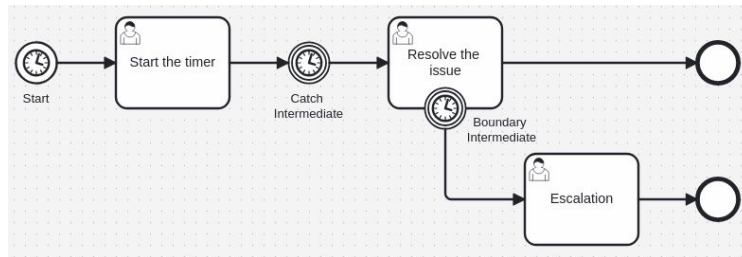
```
{
  "name": "John Wick",
  "age": 40
}
```
- ❑ Observe that the Show the person user task form was correctly filled with the obtained data
- ❑ Complete the task to finish the process

Example #3: Timers

- ❑ Timer nodes are useful for situations when our domain relies on specific time events, durations or deadlines
 - ❑ We need to create a new process instance on a specific date or in specific intervals (Start)
 - ❑ A product is painted and paint needs to dry before moving onto another stage in a factory (Catch intermediate)
 - ❑ A task has to be completed in 24 hours, otherwise we need to escalate to a manager (Boundary intermediate)
- ❑ ISO8601 format for specifying time events

Example #3: Timers

- ❑ Create the following process definition to understand how timers work



- ❑ Configure timer durations
 - ❑ Start - R3/PT5S (fire every 5 seconds upon deployment, 3 times in total)
 - ❑ Catch Intermediate - PT10S (fire once after 10 seconds)
 - ❑ Boundary Intermediate - PT10S
- ❑ After you deploy the process, every 5 seconds a new process instance will be created 3 times in a row.
- ❑ Complete **Start the timer** user task to trigger **Catch Intermediate** timer. After 10 seconds **Resolve the issue** user task is created. **Boundary Intermediate** timer is triggered and you have 10 seconds to complete the task and finish the process, otherwise **Resolve the issue** task is canceled and **Escalation** user task is created. Try both scenarios to fully understand how it works.

Example #4: Reusable Subprocess/Call Activity

- ❑ Create the following process definition called **MainProcess**



- ❑ Create a form **PersonForm** with fields:
 - ❑ **Name** of type **Text field** with key **person.name**
 - ❑ **Age** of type **Number** with key **person.age**
- ❑ Assign this form as a start form of the **MainProcess** and **Print output** user task
- ❑ Configure Inputs and Outputs for **Call a subprocess** call activity
- ❑ Configure Inputs for **Print output** user task the same way as for call activity, Outputs are not needed

The image shows a configuration panel for a BPMN task, likely for the 'Call a subprocess' or 'Print output' task. It is divided into two main sections: 'Inputs' and 'Outputs'.

Inputs:

- There is a dropdown menu showing 'person' with a plus icon and a '1' in a circle.
- Below the dropdown, there is a section for 'Local variable name' with a text field containing 'person'.
- Below that, there is a section for 'Variable assignment value' with a text field containing '= person'.
- At the bottom of the 'Inputs' section, there is a label 'Output propagation' with a right-pointing arrow.

Outputs:

- There is a dropdown menu showing 'person' with a plus icon and a '1' in a circle.
- Below the dropdown, there is a section for 'Process variable name' with a text field containing 'person'.
- Below that, there is a section for 'Variable assignment value' with a text field containing '= person'.

Example #4: Reusable Subprocess/Call Activity

- ❑ Create the process definition called **SubProcess**



- ❑ Assign previously created **PersonForm** to **Modify Person** user task
- ❑ Configure Inputs and Outputs for **Modify Person** user task

Inputs + 1 ▾

person

Local variable name

person

Variable assignment value *fx*

= person

Output propagation >

Outputs + 1 ▾

person

Process variable name

person

Variable assignment value *fx*

= person

- ❑ Go back to **MainProcess** and fill in **SubProcess ID** in **Call a subprocess Called element Process ID** field
 - ❑ This will instruct engine which subprocess to start
- ❑ Deploy and run **MainProcess**. Provide name and age as an input for **MainProcess** and start it. **MainProcess** will pass this data to **SubProcess**, which will then create **Modify Person** user task. In this task, you can change the data. After the task is completed, **SubProcess** ends and control will be returned to **MainProcess** which will take data from **SubProcess** and pass it further to **Print output** user task where you can verify that the data was correctly changed in **SubProcess**. All of this is possible thanks to the Input/Output mappings which were configured.

Running independent “subprocess” using signals

- ❑ Calling process (main process) always waits for the subprocess to finish before continuing
- ❑ If you want to trigger subprocess and continue with main process, you should use signals. Subprocess can have one “normal” start node and one (or more) signal start event node at the same time, so it can be started manually and using a signal too.
- ❑ Main process can even pass data using a signal to “subprocess” started by signal start event.
- ❑ Signal throw event (in main process)
 - ❑ You can use input mappings to define which data will be sent together with the signal.
- ❑ Signal catch event/Signal start event (“subprocess”)
 - ❑ By default, all signal variables are merged into the process instance. This behavior can be customized by defining an output mapping at the signal catch event.

THANK YOU



linkedin.com/company/red-hat



youtube.com/user/RedHatVideos



facebook.com/redhatinc



twitter.com/RedHat